

Recommender System for User Item Interactions

By

Sai Harshitha Peddi-018222087

Hemanth Sai Kosari-018223049

Bhargava Phaneendra Kumar Suryadevara-018176379

Motivation

Why this project matters

- Most real-world platforms (YouTube, Spotify, Amazon) rely heavily on *implicit feedback* (clicks, views, listens).
- Unlike explicit ratings, implicit interactions do not directly tell us preference strength.
- This makes the learning problem harder and requires specialized algorithms.

Project Motivation

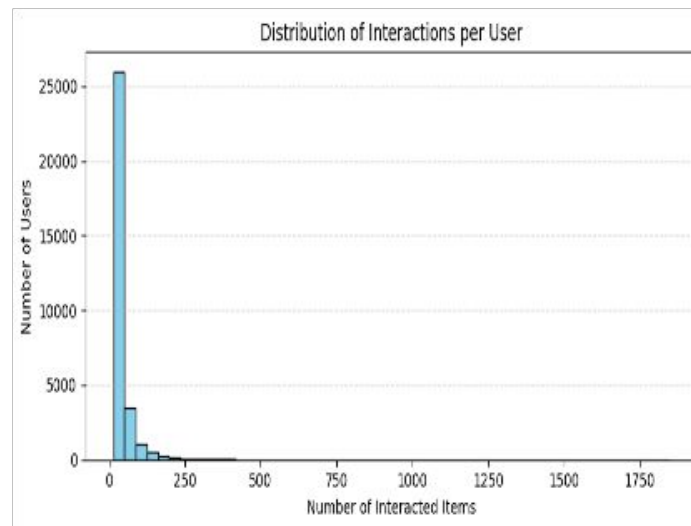
- This project evaluates three fundamentally different recommendation approaches - LightGCN, ALS, and Collaborative Filtering—each implemented independently to understand how contrasting modeling strategies behave on the *same* implicit-feedback dataset.
- By testing them under identical conditions, we can fairly compare how well each method handles sparsity, missing data, and the absence of explicit ratings.

Our objective

- Build a top-20 recommendation system using implicit interactions.
- Analyze which modeling approach works best on our dataset.

Dataset Overview:-

DATASET SUMMARY REPORT	
Total Users:	31,668
Total Items:	38,048
Total Interactions:	1,237,259
Matrix Density:	0.00102685
Matrix Sparsity:	99.8973%
User Interaction Statistics	
Users with >0 interactions:	31,668
Min interactions per user:	16
Mean interactions per user:	39.07
Median interactions per user:	25.00
Max interactions per user:	1848
Item Popularity Statistics	
Items with >0 interactions:	38,048
Min interactions per item:	1
Mean interactions per item:	32.52
Median interactions per item:	17.00
Max interactions per item:	1258



Algorithms Selected

LightGCN

LightGCN is a lightweight graph-based collaborative filtering model. It was selected because it effectively captures multi-hop user-item relationships while remaining computationally efficient. It is particularly well-suited to sparse datasets due to its propagation-based embedding refinement.

Why selected:

- Designed specifically for implicit-feedback recommendation
- Performs well on sparse user-item matrices
- Scales smoothly to large datasets
- Provides strong baseline performance without heavy neural components

ALS

ALS (Alternating Least Squares) is a latent factor model used for implicit-feedback recommendation. ALS is efficient on large, sparse datasets and offers stable, predictable performance. It generalizes beyond surface-level similarities by uncovering hidden preference patterns in the interaction matrix.

Why selected:

- Well-suited for implicit-feedback data (confidence-weighted factorization)
- Handles large, sparse user-item matrices efficiently
- Learns latent behavioral patterns beyond direct co-occurrences
- Provides strong, stable performance with straightforward tuning

CF

High sparsity → few common items → similarity is weak or unreliable. Implicit feedback noise → equal weight for all interactions leads to poor neighbor matching. Cold-start issues for users/items with few interactions. Cannot learn deep representations — relies only on surface-level similarity.

Why selected:

- It matches the dataset's structure (pure implicit, no item features).
- It provides interpretable baselines.
- It allows to test both classical similarity models and modern approaches.
- It helps analyze how simple vs. sophisticated approaches behave under sparsity

LightGCN

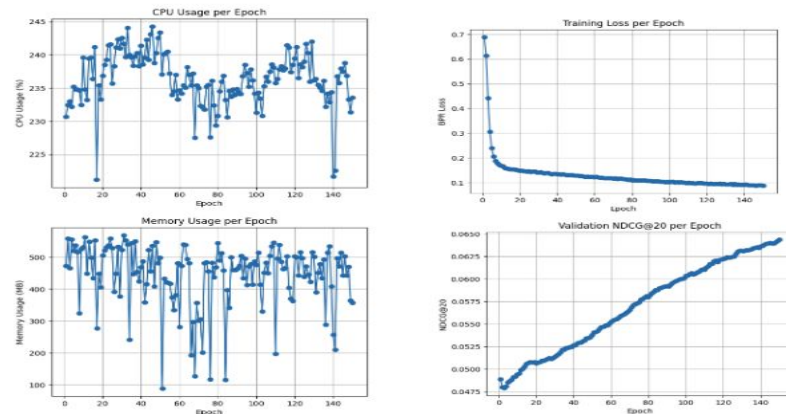
Approach:

1. Built a user-item graph from interactions
2. Performed layer-wise embedding propagation (neighbors average)
3. Combined embeddings from all layers
4. Predicted relevance using a dot product
5. Trained using BPR loss with positive/negative sampling
6. Evaluated using NDCG@20

Tweaks Tried:

- **v1:** Base LightGCN
- **v2:** Better regularization & propagation tuning
- **v3 + Popularity ($\alpha = 0.8$):** Added popularity prior (slightly worse)
- **v4 (Best):** Optimized LR, sampling, and embedding size

Top NDCG@20 = 0.0349

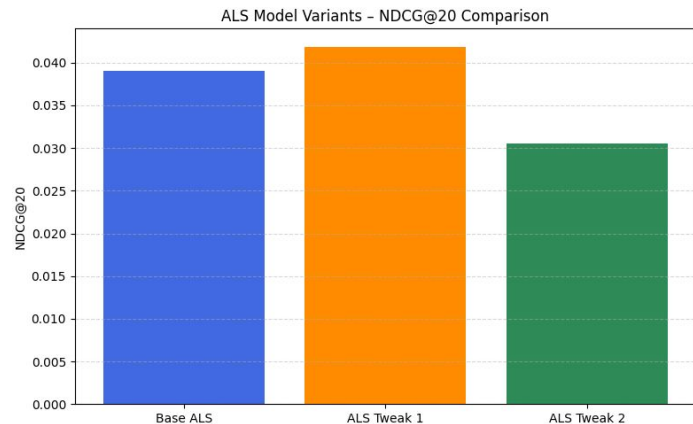


Model Variant / Submission	File Name	NDCG@20 (on 50%)
LightGCN v4 (Best-performing)	submission_lgc4.txt	0.0349
LightGCN with popularity blend ($\alpha = 0.8$)	submission_lgc3_alpha_0.8.txt	0.0322
LightGCN v2	submission-lgc-2.txt	0.0305
LightGCN v1	submission-lgc.txt	0.0284
LightGCN + popularity baseline	submission__lgc3pop.txt	0.0106

Alternating Least Squares (ALS)

- ALS is a latent factor model that learns user and item embeddings from implicit feedback using alternating optimization.
- Preprocessing included ID mapping, rare-item filtering, building a CSR sparse matrix ($\sim 31\text{K} \times \sim 38\text{K}$), and applying a per-user 90/10 train-validation split. A user_items dictionary was created to track seen items, and BM25 weighting was used only for the ALS Tweak 2 variant.
- Trained three configurations by varying latent factors (64/96/128), regularization, iterations (20–30), and using BM25 weighting for one variant.
- ALS Tweak 1 (128 factors, $\text{reg}=0.05$) achieved $\text{NDCG@20} \approx 0.0419$, the highest score among all ALS variants and all models overall.

ALS Variant	Configuration	What we Changed	NDCG
Base ALS	64 factors, $\text{reg}=0.01$, 20 iterations, $\alpha=40$	Default Setup	0.039
ALS Tweak1	128 factors, $\text{reg}=0.05$, 30 iterations, $\alpha=40$	Increased model capacity	0.0419
ALS Tweak 2	96 factors, $\text{reg}=0.03$, 25 iterations, $\alpha=20$, BM25 weighting	Normalized item popularity	0.0305



Collaborative Filtering

- Recommends items based on **similarity** between users or items. Core idea: “*Users who interacted with similar items will have similar preferences.*”
- **Item–Item CF** → recommends items similar to those the user already consumed. Build item–user matrix ; Compute cosine similarity between items ; User profile = sum of their interacted item vectors Rank items by similarity to user profile
Variants: Sum profile (best), Mean profile, IDF-weighted profile
- **Model-based CF** Reduce user–item matrix into k latent dimensions
Variants: 32 factors, 64 factors, 128 factors Results: Best validation NDCG → 128 dims

Model Variant	Result	Comments
Item–Item CF (Cosine, Sum & Mean Profiles)	0.04321	Both profiles perform identically because cosine similarity depends on vector direction, not magnitude. This makes cosine the strongest and most stable CF variant.
Item–Item CF (Cosine, IDF-Weighted)	0.027495	Down-weighting popular items removed meaningful collaborative signal, leading to slightly weaker performance.
Truncated SVD (32 Components)	0.022373	Lower-dimensional embedding underfits interactions.
Truncated SVD (64 Components)	0.023711	Best SVD variant; good balance between variance captured and overfitting.
Truncated SVD (128 Components)	0.026600	Higher dimension introduces noise and does not improve ranking.

Comparison of Models

LightGCN vs. ALS

- While LightGCN effectively captured multi-hop user–item relationships, ALS achieved better overall prediction on this dataset due to its strong latent factor modeling.
- ALS handled the dataset’s extreme sparsity more robustly by leveraging confidence-weighted matrix factorization, resulting in more stable and consistent embeddings.
- LightGCN benefited from graph propagation but was more sensitive to hyperparameters and required deeper tuning, whereas ALS converged faster and produced higher-quality recommendations with simpler optimization.
- Overall, although LightGCN is efficient and modern, ALS outperformed it on this dataset in terms of ranking quality and generalization.

Collaborative Filtering vs. ALS

- **Handling Sparsity:** CF struggles when users have few overlapping interactions, while ALS learns latent factors and performs well even in extremely sparse datasets.
- **Model Capability:** CF relies on direct similarity, which is simple but limited; ALS captures deeper hidden patterns by factorizing the entire user–item matrix.
- **Accuracy:** ALS achieved significantly higher NDCG@20 in our experiments, whereas CF produced the lowest scores due to sparse co-occurrence.
- **Scalability & Stability:** ALS scales efficiently and converges reliably on large datasets, while CF becomes less reliable and more sensitive to noise as sparsity increases.

Conclusion

- The comparison of LightGCN, ALS, and Collaborative Filtering shows that each method brings different strengths to handling sparse implicit-feedback data, with performance depending strongly on how well the model aligns with the dataset's structure.
- ALS achieved the best overall results, offering stable convergence, effective handling of sparsity, and strong ranking performance due to its confidence-weighted matrix factorization approach.
- LightGCN captured multi-hop user-item relationships efficiently but was more sensitive to hyperparameter choices and required deeper tuning to match ALS performance.
- Collaborative Filtering provided a simple and interpretable baseline, but its neighborhood-based assumptions limited its ability to capture deeper user-item dependencies in highly sparse environments.

Things That Didn't Work Well

- Jaccard similarity - underperformed Too few overlapping users → weak similarity signal Much slower than cosine on large item sets
- Over-normalization harmed performance
- Early versions underperformed Initial LightGCN v1/v2 variants were weaker until proper tuning
- High latent dimensions introduced noise Increasing factors beyond optimal size did not improve results